جامعة الحسن الثاني بالدار البيضاء
كلية العلوم بن مسيك

FSBM

FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Rapport de Projet : Automatisation d'Infrastructure Cloud et Pipeline CI/CD

Réalisé par : Abderrahmane HASSANI & Anas ELHAFI

Résumé

Ce rapport détaille la conception, le provisionnement et l'automatisation complète d'une infrastructure Cloud couplée à un pipeline d'Intégration et de Déploiement Continus (CI/CD). L'objectif principal de ce projet est de moderniser le cycle de vie d'une application web (Portfolio) en substituant les interventions manuelles par une approche rigoureuse basée sur l'Infrastructure as Code (IaC) et la gestion de configuration.

Déployée sur Amazon Web Services (AWS) via des instances Debian 12, l'architecture est instanciée dynamiquement par Terraform et orchestrée de manière centralisée par Ansible. La gestion des charges de travail conteneurisées est assurée par un cluster Kubernetes allégé (K3s), tandis qu'un serveur Jenkins pilote l'automatisation des phases de compilation (via Docker BuildKit), de publication sécurisée sur Docker Hub, et de mise en production continue.

Une attention particulière a été portée à la résilience et à l'optimisation des environnements contraints en ressources (instances t3.micro). Des mécanismes avancés d'auto-remédiation (*self-healing*) ont été modélisés pour pallier la volatilité des adresses IP publiques, assurant ainsi la tolérance aux pannes du cluster. *In fine*, ce projet démontre la viabilité d'une chaîne DevOps de bout en bout : sécurisée, reproductible et capable d'assurer des mises à jour applicatives sans interruption de service.

Sommaire

1. Introduction et Aperçu du Projet 3
2. Provisionnement de l'Infrastructure Cloud 3
2.1. Sélection de l'Infrastructure Matérielle et du Système d'Exploitation 3
2.2. Configuration Réseau et Gestion des Flux de Trafic 3
2.3. Provisionnement Automatisé des Instances EC2 4
3. Défis Techniques et Solutions Apportées 4
4. Gestion de Configuration et Orchestration 5
4.1. Centralisation de l'Orchestration et Gestion de Configuration via Ansible 5
4.2. Interopérabilité entre l'Infrastructure as Code et la Gestion de Configuration 5
4.3. Exécution Séquentielle et Ciblée des Tâches d'Orchestration 5
5. Résilience de l'Infrastructure 6
6. Intégration et Déploiement Continus 6
6.1. Configuration de l'Orchestrateur d'Intégration 6
6.2. Modélisation et Exécution du Déploiement (Kubernetes) 7
7. Guide de Reproduction de l'Infrastructure 7
7.1. Génération des vecteurs d'authentification 7
7.2. Préparation de l'environnement local et sécurisation de la clé 8
7.3. Déploiement via Terraform 8
7.4. Configuration Post-Déploiement de l'Orchestrateur Jenkins 10
A. Initialisation et Intégration des Extensions 10
B. Gestion Sécurisée des Habilitations 10
C. Interfaçage avec le Système de Contrôle de Version Github 11
D. Automatisation du Déclenchement via Webhook 11
7.5. Amorçage et Supervision du Déploiement 12
8. Annexes et Référentiels de Code Source 13
8.1. Organisation Structurelle du Monorepo 13
8.2. Accès aux Sources 13

1. Introduction et Aperçu du Projet

L'objectif de ce projet était de concevoir, d'approvisionner et d'automatiser un pipeline d'Intégration et de Déploiement Continus (CI/CD) complet pour héberger une application web conteneurisée. Nous avons opéré une transition depuis une configuration manuelle des serveurs vers un paradigme entièrement automatisé, reposant sur l'Infrastructure as Code (IaC) et la Gestion de Configuration.

À l'aide de Terraform, nous avons défini notre infrastructure cloud sur Amazon Web Services (AWS). Ansible a ensuite été utilisé pour orchestrer la configuration logicielle sur plusieurs nœuds. Enfin, en combinant Jenkins et K3s (une distribution Kubernetes allégée), nous avons bâti un pipeline de déploiement capable de conteneuriser l'application, de la publier sur Docker Hub, et de mettre à jour le cluster de production sans aucune intervention manuelle.

2. Provisionnement de l'Infrastructure Cloud

2.1. Sélection de l'Infrastructure Matérielle et du Système d'Exploitation

Pour héberger notre architecture, le choix s'est porté sur des instances AWS de type t3.micro. Ces instances dites « burstables » (à capacité extensible) constituent un socle particulièrement économique, parfaitement adapté au déploiement de microservices et aux environnements de test CI/CD. Côté système d'exploitation, nous avons privilégié l'AMI officielle Debian 12. Appréciée pour sa stabilité à toute épreuve, son empreinte mémoire extrêmement réduite et l'absence de paquets superflus par rapport à d'autres distributions, Debian s'impose comme une fondation allégée idéale pour faire tourner efficacement un cluster K3s.

2.2. Configuration Réseau et Gestion des Flux de Trafic

Afin de sécuriser et d'orchestrer les flux réseau, un Groupe de Sécurité AWS (Security Group) a été défini de manière déclarative via Terraform pour faire office de pare-feu virtuel. Les règles de trafic entrant (ingress) ont été méticuleusement cartographiées pour répondre aux exigences strictes de notre architecture :

- Port 22 (SSH) : Réservé à l'accès administratif distant et sécurisé pour l'exécution d'Ansible et de Terraform.
- Port 8080 (TCP) : Ouvert pour rendre accessible l'interface d'administration web du serveur Jenkins.
- Port 6443 (TCP) : Dédié à l'autorisation des communications (internes et externes) avec l'API Kubernetes.
- Port 30080 (TCP) : Configuré en tant que NodePort Kubernetes, son rôle est d'exposer publiquement l'application web finale sur Internet.

- Règle d'auto-référence : Une autorisation globale a été instaurée pour l'ensemble du trafic interne entre les instances rattachées à ce groupe de sécurité. Cela garantit une communication inter-nœuds K3s totalement fluide et sans entrave.

2.3. Provisionnement Automatisé des Instances EC2

Afin de rompre avec l'utilisation manuelle de la console web AWS, l'ensemble de nos serveurs (Jenkins, nœud Master et nœud Worker) a été modélisé sous forme de ressources Terraform. Pour parfaire la préparation des machines dès leur phase d'amorçage, l'attribut user_data a été exploité de manière stratégique. Ce dernier permet d'injecter des scripts Bash qui s'exécutent au tout premier démarrage du système d'exploitation, automatisant ainsi le formatage et le montage instantané de la partition Swap vitale pour nos instances.

3. Défis Techniques et Solutions Apportées

La construction d'un pipeline entièrement automatisé et sensible à l'état de l'infrastructure a soulevé plusieurs défis architecturaux majeurs, à commencer par la gestion stricte des ressources matérielles. Si l'utilisation d'instances AWS t3.micro permet de réduire considérablement les coûts, elle impose des limites techniques importantes avec seulement 2 vCPU "burstables" et 1 Go de RAM. L'exécution simultanée du serveur Jenkins, des compilations d'images Docker particulièrement gourmandes en ressources et des agents K3s a rapidement provoqué un goulot d'étranglement sévère. Le processeur atteignait sa limite de crédit, entraînant un phénomène de CPU throttling, tandis que la mémoire vive saturait. Cette surcharge provoquait le gel des instances (freeze) ou le déclenchement du mécanisme de protection du système d'exploitation, supprimant de force les processus critiques (erreurs OOM-Killer).

Pour pallier ces limitations, une stratégie d'optimisation à trois niveaux a été mise en place. Côté infrastructure, l'ajout dynamique d'un fichier d'échange (Swap) de 1 Go via les scripts user_data de Terraform a fourni une mémoire de secours vitale pour encaisser les pics de charge imposés par Jenkins. Côté intégration et déploiement continus (CI/CD), l'activation du nouveau moteur via la variable DOCKER_BUILDKIT=1 a drastiquement réduit l'empreinte CPU et mémoire lors de la construction des conteneurs. Enfin, au niveau de l'orchestration, des limites strictes ont été définies dans les manifestes Kubernetes (resources.limits: memory: "256Mi", cpu: "250m") pour garantir que les pods de l'application ne monopolisent pas les ressources limitées du nœud Worker.

Un second défi technique critique concernait la problématique des adresses IP éphémères et la synchronisation de l'état. AWS attribuant par défaut des adresses IP publiques temporaires, l'arrêt régulier des instances pour des raisons de maîtrise des coûts entraînait inévitablement un changement d'IP à chaque redémarrage. Par conséquent, Terraform échouait en tentant de se connecter via SSH à des adresses obsolètes, ou subissait l'erreur "Unknown After Apply" lorsque AWS n'avait pas encore finalisé l'attribution des nouvelles IP.

Pour résoudre ce problème de synchronisation, l'automatisation a été scindée en deux déclencheurs (triggers) Terraform distincts. La première phase, initial_setup, a été strictement liée aux identifiants immuables des instances AWS afin de ne s'exécuter que lors de la création initiale de l'infrastructure. Un second déclencheur, reconfigure_stack, a été directement lié aux adresses IP publiques. Cette architecture permet à Terraform de détecter élégamment les changements d'IP après chaque cycle d'arrêt et de démarrage, déclenchant ainsi l'exécution d'un playbook Ansible spécifique destiné à restaurer la communication et à reconfigurer le cluster de manière totalement transparente.

4. Gestion de Configuration et Orchestration

4.1. Centralisation de l'Orchestration et Gestion de Configuration via Ansible

Dans la perspective d'éliminer toute intervention manuelle inhérente au déploiement logiciel, une architecture asymétrique a été adoptée, au sein de laquelle le nœud maître Kubernetes (Master Node) a été désigné comme nœud de contrôle Ansible (Control Node). Cette approche permet de centraliser la gestion de configuration au plus près de l'orchestrateur de conteneurs.

4.2. Interopérabilité entre l'Infrastructure as Code et la Gestion de Configuration

L'intégration de ces deux paradigmes s'opère dès la phase d'amorçage (provisioning). Le script d'initialisation user_data du nœud maître automatise l'installation d'Ansible par l'intermédiaire du gestionnaire de paquets apt-get. Concomitamment, Terraform procède à la génération dynamique d'un fichier d'inventaire (hosts.ini) sur l'environnement d'exécution local. Ce fichier répertorie en temps réel les nouvelles adresses IP, tant publiques que privées, allouées par l'infrastructure cloud AWS. À l'issue de cette génération, Terraform assure le transfert sécurisé de cet inventaire, de la clé cryptographique asymétrique (.pem), ainsi que des scripts de configuration (playbooks YAML) vers le nœud maître.

4.3. Exécution Séquentielle et Ciblée des Tâches d'Orchestration

L'orchestration logicielle est par la suite déléguée localement au nœud maître. En exploitant l'inventaire hosts.ini, Ansible identifie précisément les instances cibles afin d'exécuter les tâches spécifiques à chaque rôle architectural :

- master_setup.yml : Restreint au groupe d'hôtes [master], ce playbook supervise l'installation du service K3s et génère le jeton cryptographique (token) requis pour l'authentification et l'enrôlement ultérieur des autres nœuds au sein du cluster.
- jenkins_setup.yml : Affecté au groupe [jenkins], ce script automatise le déploiement de l'environnement d'exécution Java, du démon Docker et du serveur d'intégration continue

Jenkins. Il procède également à la copie sécurisée du fichier de configuration kubeconfig depuis le nœud maître, conférant ainsi à Jenkins les privilèges d'administration requis pour interagir avec l'API Kubernetes.

- worker_setup.yml : Destiné exclusivement au groupe [worker], ce processus configure l'environnement d'exécution de conteneurs Docker et initialise l'agent K3s. L'attachement au cluster est finalisé en exploitant l'adresse IP privée du nœud maître, assurant de facto une communication réseau isolée et sécurisée.

5. Résilience de l'Infrastructure

La rationalisation des coûts opérationnels inhérents au cloud computing implique la mise hors tension régulière des instances de calcul. Afin de pallier les limitations induites par la volatilité des adresses IP publiques lors des redémarrages successifs, un mécanisme d'auto-remédiation (self-healing) a été modélisé et intégré à l'architecture.

Lorsqu'un cycle de redémarrage entraîne l'allocation de nouvelles adresses IP, Terraform identifie cette divergence d'état par l'analyse comparative de son fichier d'état local (tfstate) et de l'infrastructure distante. Afin d'éviter l'exécution redondante et potentiellement destructrice des routines d'initialisation de base, le système génère de manière déterministe un nouvel inventaire hosts.ini et déclenche l'exécution d'un playbook de remédiation dédié (reconfigure.yml).

Cette procédure assure la mise à jour des paramètres de l'agent K3s hébergé sur le nœud travailleur (worker) et du fichier kubeconfig du serveur Jenkins, en les reconfigurant pour cibler la nouvelle adresse IP privée du nœud maître. Ce paradigme technique garantit la restauration immédiate et transparente de la cohésion topologique du cluster, et ce, en l'absence de toute intervention humaine.

6. Intégration et Déploiement Continus

Suite à la validation opérationnelle et à la sécurisation de l'infrastructure sous-jacente, l'implémentation du pipeline d'intégration et de déploiement continus a été finalisée. Ce pipeline constitue la clé de voûte de l'automatisation des mises en production.

6.1. Configuration de l'Orchestrateur d'Intégration

Le serveur Jenkins a été interfacé avec le système de contrôle de version (dépôt GitHub) hébergeant le code source de l'application. Afin de permettre l'interaction avec le démon Docker, les extensions logicielles appropriées (Docker Pipeline plugins) ont été intégrées. Par ailleurs, les vecteurs d'authentification relatifs au registre d'images (Docker Hub) ont été provisionnés de manière chiffrée au sein du gestionnaire global des habilitations de Jenkins, garantissant ainsi la sécurité des identifiants lors des phases de téléversement.

6.2. Modélisation et Exécution du Déploiement (Kubernetes)

La logique d'orchestration logicielle a été formalisée de manière déclarative au sein d'un fichier de configuration (Jenkinsfile). Ce paradigme déclaratif structure le flux de travail en trois phases séquentielles distinctes :

- Phase d'Intégration Continue (CI) : Suite à la récupération automatisée du code source, le processus emploie le moteur de compilation avancé Docker BuildKit (DOCKER_BUILDKIT=1). Cette approche optimise significativement les temps de traitement et la consommation de ressources lors de la construction de l'image de conteneur multi-étapes (multi-stage build).
- Phase de Livraison Continue (Continuous Delivery) : L'artefact logiciel résultant est estampillé dynamiquement au moyen de l'identifiant d'exécution univoque (build number) généré par Jenkins. L'image conteneurisée est ensuite téléversée de manière sécurisée vers le registre public Docker Hub.
- Phase de Déploiement Continu (Continuous Deployment) : Tirant parti du fichier d'authentification kubeconfig préalablement provisionné par Ansible, Jenkins procède à la mutation dynamique du manifeste Kubernetes (deployment.yaml) par l'injection de la nouvelle balise d'image. La réconciliation de l'état souhaité est ensuite soumise à l'API du cluster K3s par le biais de l'utilitaire kubectl.

Finalement, le plan de contrôle Kubernetes (Control Plane du nœud maître) orchestre la planification (scheduling) du nouveau pod applicatif sur le nœud travailleur (Worker Node). L'accessibilité externe de l'application est instantanément assurée de manière transparente par l'intermédiaire d'un service réseau de type NodePort, spécifiquement configuré sur le port d'écoute 30080.

7. Guide de Reproduction de l'Infrastructure

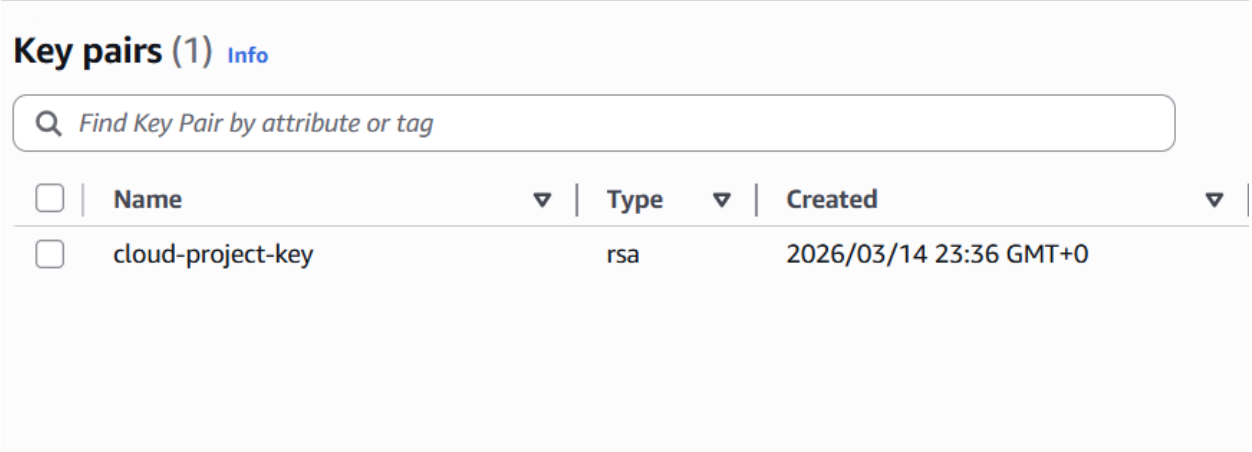
Ce projet est conçu pour être entièrement reproductible. Voici les étapes techniques et les commandes Linux nécessaires pour recréer cette architecture à partir de zéro, en partant du principe que Terraform et AWS CLI sont préalablement installés.

7.1. Génération des vecteurs d'authentification

Avant de procéder au déploiement, il est impératif de générer la clé cryptographique asymétrique qui permettra à Terraform et Ansible de s'authentifier de manière sécurisée (via le protocole SSH) auprès des futures instances EC2.

1. Connectez-vous à l'interface de gestion AWS (AWS Management Console) et naviguez vers le service EC2.
2. Dans le menu de navigation latéral gauche, sous la section Réseau et sécurité (Network & Security), sélectionnez Paires de clés (Key Pairs).

3. Cliquez sur Créer une paire de clés (Create key pair) et appliquez la configuration stricte suivante :
    - Nom : cloud-project-key (Ce nom exact est requis car il est codé en dur dans les scripts Terraform).
    - Type de paire de clés : RSA.
    - Format de fichier de clé privée : .pem (Format standard requis pour OpenSSH).
4. Cliquez sur Créer. Le fichier cloud-project-key.pem sera généré et téléchargé automatiquement sur votre poste local.



7.2. Préparation de l'environnement local et sécurisation de la clé

Une fois la clé obtenue, il convient de préparer le répertoire de travail local et d'appliquer les restrictions de sécurité exigées par les systèmes UNIX pour la manipulation des clés privées.

```
# 1. Cloner le dépôt contenant les scripts d'infrastructure (Terraform,
Ansible)
git clone https://github.com/DOWNLOAD-it/autoDeploy
cd autoDeploy

# 2. Déplacer la clé cryptographique AWS téléchargée vers la racine du projet
# (Ajustez le chemin de téléchargement selon votre système d'exploitation)
mv ~/Downloads/cloud-project-key.pem .

# 3. Restreindre les privilèges d'accès de la clé privée (Obligatoire sous
Linux/macOS)
# La commande chmod 400 garantit que le fichier est en lecture seule pour le
propriétaire
chmod 400 cloud-project-key.pem
```

7.3. Déploiement via Terraform

Le déploiement s'articule autour de l'exécution séquentielle des directives Terraform au sein de l'environnement local. Ce processus valide la topologie déclarée et orchestre le provisionnement des ressources sur le cloud AWS.Bash

```
# 1. Initialisation de l'environnement de travail
# (Téléchargement des modules et des fournisseurs AWS/Local)
terraform init

# 2. Évaluation et prévisualisation de l'état souhaité
# (Analyse syntaxique et calcul de l'arbre des dépendances)
terraform plan
```

```
C:\Users\dodso\Desktop\autoDeploy>terraform plan
data.aws_ami.debian_12: Reading...
data.aws_ami.debian_12: Read complete after 0s [id=ami-0ac375b31bb86780b]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.jenkins_server will be created
  + resource "aws_instance" "jenkins_server" {
      + ami                                  = "ami-0ac375b31bb86780b"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
```

```
# 3. Exécution du provisionnement et déclenchement de l'automatisation
Ansible
terraform apply -auto-approve
```

```
null_resource.reconfigure_stack: Creation complete after 14s [id=2234390630559573220]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

K8s-Master	i-0630eb1bc700a7f7a	Running	t3.micro	3/3 checks passed	View alarms +	eu-west-3c
K8s-Worker	i-006804d9d42c07e1a	Running	t3.micro	3/3 checks passed	View alarms +	eu-west-3c
Jenkins-Server	i-00a8d1694871f5fd8	Running	t3.micro	3/3 checks passed	View alarms +	eu-west-3c

À l'issue de la phase d'amorçage, il est impératif de procéder à une vérification d'intégrité du plan de contrôle (Control Plane) et de l'état d'enrôlement des nœuds de calcul. Cette validation s'effectue via l'établissement d'une session distante sécurisée vers le nœud maître.

```
# 1. Établissement de la connexion SSH vers le nœud maître
# (L'adresse IP publique est restituée par la sortie standard de Terraform)
ssh -i "cloud-project-key.pem" admin@<ADRESSE_IP_PUBLIQUE_DU_MASTER>

# 2. Interrogation de l'API Kubernetes pour valider la topologie du cluster
# (Les nœuds Master et Worker doivent impérativement afficher le statut
"Ready")
sudo kubectl get nodes
```

```
admin@ip-172-31-40-10:~$ sudo kubectl get nodes
NAME                STATUS   ROLES           AGE     VERSION
ip-172-31-39-255    Ready    <none>          7m47s   v1.34.5+k3s1
ip-172-31-40-10     Ready    control-plane   10m     v1.34.5+k3s1
```

```
# 3. Clôture de la session distante
exit
```

7.4. Configuration Post-Déploiement de l'Orchestrateur Jenkins

Bien que l'infrastructure sous-jacente soit opérationnelle et que les dépendances logicielles (Docker, Java, Jenkins) aient été provisionnées par Ansible, l'orchestrateur nécessite une configuration applicative pour interagir avec les référentiels externes.

A. Initialisation et Intégration des Extensions

Lors de la première connexion à l'interface d'administration de Jenkins (http://<ADRESSE_IP_PUBLIQUE_JENKINS>:8080), il convient de déverrouiller le système et d'installer les modules d'extension indispensables à l'exécution du pipeline.

1. Récupération du mot de passe d'administration initial via une session SSH sur le serveur Jenkins :

```
null_resource.initial_setup (remote-exec): ok: [jenkins_server] => {
null_resource.initial_setup (remote-exec):     "msg": [
null_resource.initial_setup (remote-exec):         "*************************************************************",
null_resource.initial_setup (remote-exec):         "JENKINS INSTALLATION COMPLETE",
null_resource.initial_setup (remote-exec):         "Access URL: http://13.39.111.79:8080",
null_resource.initial_setup (remote-exec):         "Admin Password: 58f462b73d1a454580517dac185d18cc\n",
null_resource.initial_setup (remote-exec):         "*************************************************************"
null_resource.initial_setup (remote-exec):     ]
null_resource.initial_setup (remote-exec): }
```

2. Installation des extensions requises depuis le gestionnaire de plugins:
   - ✓ Docker Pipeline et Docker Plugin : Essentiels pour l'exécution des commandes de conteneurisation au sein du Jenkinsfile.
   - ✓ Pipeline et Git Plugin : Requis pour la récupération du code source et l'interprétation du paradigme déclaratif.

B. Gestion Sécurisée des Habilitations

Pour garantir la sécurité des accès aux plateformes tierces sans exposer les identifiants en clair dans le code source, les vecteurs d'authentification doivent être stockés dans le coffre-fort chiffré de Jenkins (Manage Jenkins > Credentials > System > Global credentials).

1. Authentification Docker Hub :
   - ✓ Type : Username with password
   - ✓ Username : docker_user (Votre identifiant Docker Hub)
   - ✓ Password : jeton d'accès personnel (Personal Access Token).
   - ✓ ID : docker-hub-credentials (Cet identifiant exact est appelé de manière déclarative à l'étape "Push to Docker Hub" de votre Jenkinsfile).

Description	Scope	Status	Source	Created	Last used	Expiration date
Jenkins-CI-CD	Read, Write, Delete	Active	Manual	Mar 23, 2026 at 13:31:31	Never	Never

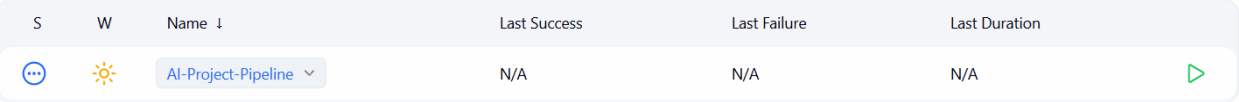
Rows per page: 10 1–1 of 1

2. Authentification GitHub (Optionnelle pour les dépôts publics, recommandée pour les dépôts privés) :
   - ✓ Type : Secret text (pour un Personal Access Token) ou SSH Username with private key.
   - ✓ ID : github-credentials

C. Interfaçage avec le Système de Contrôle de Version Github

La création de la tâche d'automatisation (Job) permet de lier Jenkins au référentiel hébergeant l'application Portfolio.

1. Depuis le tableau de bord Jenkins, sélectionnez New Item, nommez le projet (ex: Portfolio-CICD), et choisissez le type Pipeline.
2. Dans la section Pipeline, configurez la définition sur Pipeline script from SCM (Gestionnaire de Code Source).
3. SCM : Sélectionnez Git.
4. Repository URL : Saisissez l'URL de votre dépôt (ex: https://github.com/DOWNLOAD-it/portfolio.git).
5. Script Path : Indiquez Jenkinsfile (assurez-vous que ce fichier est bien présent à la racine de votre dépôt).

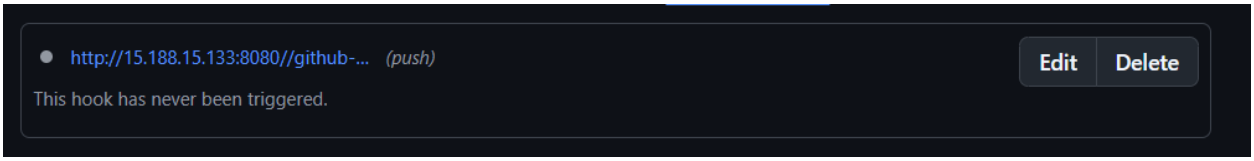


D. Automatisation du Déclenchement via Webhook

Afin de concrétiser le principe d'Intégration Continue (CI), le pipeline ne doit pas être déclenché manuellement, mais réagir dynamiquement à chaque modification du code source.

Côté Jenkins : Dans la configuration de votre Pipeline, cochez l'option GitHub hook trigger for GITScm polling.

1. Côté GitHub : Naviguez vers les paramètres de votre dépôt (Settings > Webhooks).
2. Cliquez sur Add webhook et configurez les paramètres suivants :
   - Payload URL : http://<ADRESSE_IP_PUBLIQUE_JENKINS>:8080/github-webhook/ (N'oubliez pas le slash final).
   - Content type : application/json.
   - Événements déclencheurs : Sélectionnez Just the push event.
3. Validez en cliquant sur Add webhook. Un ping de test sera envoyé par GitHub pour confirmer l'établissement de la communication bidirectionnelle.



7.5. Amorçage et Supervision du Déploiement

L'architecture logicielle et matérielle est désormais intégralement couplée.

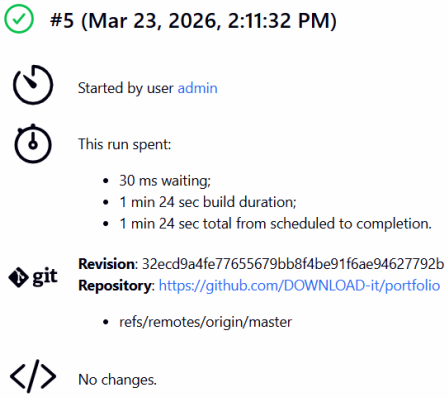
Déclenchement Manuel (Test de validation) : Depuis l'interface Jenkins, cliquez sur Build Now pour initier la première exécution. Cette étape permet de valider le bon fonctionnement de la chaîne (clonage, compilation BuildKit, téléversement sur Docker Hub, et mutation du manifeste K3s).

Déclenchement Automatisé (Workflow de Production) :

```
# Depuis votre machine locale, simulez une mise à jour du code
git commit -am "Mise à jour de l'interface du portfolio et test du Webhook"
git push origin main
```

```
abderrahmane@HASSANI MINGW64 ~/Desktop/Cloud app (master)
$ git push origin master
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 6.00 KiB | 2.00 MiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/DOWNLOAD-it/portfolio.git
   d5180d9..197f0cd  master -> master
```

Dès la réception de la commande push, GitHub transmettra une charge utile (payload) au serveur Jenkins. L'orchestrateur instanciera immédiatement un nouvel environnement de compilation, produira l'artefact Docker estampillé du nouveau numéro de build, et appliquera la mise à jour sur le cluster Kubernetes.



8. Annexes et Référentiels de Code Source

L'intégralité du cycle de vie du projet — du provisionnement de l'infrastructure Cloud au déploiement de l'application finale — est centralisée au sein d'un dépôt unique (Monorepo). Cette approche garantit une traçabilité rigoureuse entre les ressources d'infrastructure et les versions applicatives, tout en respectant le principe de séparation des préoccupations (Separation of Concerns).

8.1. Organisation Structurelle du Monorepo

Bien que regroupée dans un référentiel unique, la base de code est organisée de manière modulaire afin d'isoler les responsabilités techniques :

- Composante Applicative (/app) : Contient le code source de l'application web Portfolio, ainsi que la définition de son environnement d'exécution conteneurisé (Dockerfile).
- Composante Orchestration Kubernetes (/k8s) : Centralise les manifestes déclaratifs (Deployment, Service) nécessaires au pilotage des conteneurs sur le cluster.
- Infrastructure as Code (/terraform) : Regroupe les scripts HCL dédiés au provisionnement des instances EC2 (Debian 12) et à la configuration réseau sur le cloud AWS.
- Gestion de Configuration (/ansible) : Isole les playbooks automatisant l'installation de la solution K3s, la configuration de Docker et la sécurisation inter-nœuds.
- Pilotage CI/CD (Jenkinsfile) : Fichier pivot situé à la racine, définissant la logique d'orchestration globale du pipeline de livraison.

8.2. Accès aux Sources

Le code source complet, incluant l'infrastructure et l'applicatif, est hébergé sur la plateforme GitHub :

Référentiel Centralisé : https://github.com/DOWNLOAD-it/portfolio/tree/master